FACULTAD DE INGENIERÍA | PROGRAMA DE INGENIERÍA DE SISTEMAS

SIMULADOR DE PRUEBAS SABER TyT: SEGURIDAD JWT, AUTORIZACIÓN POR ROLES Y EVALUACIÓN DE CUESTIONARIOS EN DJANGO

Informe Técnico de Implementación Práctica — Segundo Parcial Evaluativo

Jenkner Johan Arteaga Delgado

Neider Duvan Quindigua Machoa

David Mauricio Zambrano Guerrero

Facultad de Ingeniería, Programa de Ingeniería de Sistemas

Institución Universitaria del Putumayo (IUP)

Electiva Profesional III – Frameworks de Aplicaciones Web

Docente Titular: Ing. Raúl Rodríguez

Mocoa, Putumayo, Colombia

FACULTAD DE INGENIERÍA | PROGRAMA DE INGENIERÍA DE SISTEMAS

Octubre de 2026

Resumen

El presente informe técnico expone la arquitectura, desarrollo e integración del sistema integral de evaluación para pruebas de Estado Saber TyT, desarrollado bajo el framework web Django y MariaDB/MySQL. El proyecto consolida la continuidad del taller formativo progresivo en la asignatura Frameworks de Aplicaciones Web, incorporando una infraestructura de seguridad basada en Tokens Web JSON (JWT) con rotación y revocación en lista negra, un modelo de control de acceso por roles dinámicos (RBAC) con permisos granulares, un banco de preguntas clasificado por competencias institucionales con detección de duplicados mediante la función criptográfica SHA-256, un motor de importación masiva de reactivos mediante hojas de cálculo Excel (.xlsx), y un subsistema de evaluación cronometrada en tiempo real con calificación automática y retroalimentación pedagógica calculada exclusivamente en el servidor. Las pruebas ejecutadas demuestran el cumplimiento del 100% de los lineamientos establecidos en la guía evaluativa del Segundo Parcial de la Institución Universitaria del Putumayo.

Palabras clave: Django, JSON Web Tokens (JWT), Saber TyT, control de acceso por roles (RBAC), evaluación automatizada, huella digital SHA-256, MariaDB.

Tabla de Contenido

1. Introducción y Contexto Institucional **4**
2. Objetivos del Informe Técnico **5**
   2.1 Objetivo General **5**
   2.2 Objetivos Específicos **5**
3. Arquitectura Global de la Solución **6**
   3.1 Patrón de Arquitectura MVT y Componentes Backend **6**
   3.2 Base de Datos Relacional y Esquema Físico **7**
4. Módulo 1: Autenticación Robusta y Seguridad con Tokens JWT **8**
   4.1 Ciclo de Vida del Token: Acceso, Renovación y Revocación **8**
   4.2 Restricción y Decoradores de Seguridad Verificados en el Servidor **9**
   4.3 Control de Inactividad de Cuentas Institucionales **10**
5. Módulo 2: Administración de Usuarios y Roles Configurables (RBAC) **11**
   5.1 Gestión de Cuentas de Usuario y Perfiles Académicos **11**
   5.2 Configuración Dinámica de Roles y Matriz de Permisos **12**
   5.3 Caso de Uso: Demostración del Rol Docente Evaluador **13**
6. Módulo 3: Banco de Preguntas y Clasificación Pedagógica **14**
   6.1 Organización por Competencias Oficiales Saber TyT **14**
   6.2 Operaciones CRUD: Registro, Consulta, Edición y Activación **15**
   6.3 Prevención de Duplicados Mediante Huella Digital SHA-256 ..... **16**
7. Módulo 4: Importación Masiva de Reactivos desde Excel **17**
   7.1 Estructura de la Plantilla Reglamentaria y Mapeo **17**
   7.2 Algoritmo de Validación Atómica en Dos Fases **18**
8. Módulo 5: Parametrización y Disponibilidad de Cuestionarios **19**
   8.1 Creación y Asociación Ponderada de Reactivos **19**
   8.2 Validación Estricta de Temporalidad e Intentos en Backend **20**
9. Módulo 6: Presentación de la Prueba en Vivo y Calificación **21**
   9.1 Experiencia Interactiva del Estudiante y Temporizador **21**

9.2 Protección Antifraude y Ocultamiento de Respuestas Correctas **22**
9.3 Algoritmo de Calificación Automática y Aislamiento de Intentos **23**
10. Matriz de Cumplimiento de la Rúbrica Evaluativa **24**
11. Conclusiones y Recomendaciones **25**
12. Referencias Bibliográficas **26**

Índice de Figuras

Figura 1. Portada e Interfaz Pública del Simulador Saber TyT **5**
Figura 2. Interfaz de Inicio de Sesión Institucional con Validación JWT **6**
Figura 3. Panel de Control de Administración con Métricas y Accesos Rápidos **7**
Figura 4. Módulo de Gestión de Usuarios y Filtros Institucionales **9**
Figura 5. Modal Interactivo para Edición de Usuario Institucional **10**
Figura 6. Configuración de Roles y Asignación Granular de Permisos **11**
Figura 7. Banco de Preguntas con Filtros por Competencia y Tipo **13**
Figura 8. Modal de Edición de Reactivo con Recálculo de Huella SHA-256 ..... **14**
Figura 9. Interfaz de Importación Masiva de Preguntas desde Excel **16**
Figura 10. Configuración Reglamentaria de Cuestionario y Preguntas Asignadas **18**
Figura 11. Dashboard del Estudiante con Cuestionarios Disponibles y Estadísticas **20**
Figura 12. Reproductor de Evaluación en Vivo con Temporizador Digital Regresivo **21**
Figura 13. Resultado y Calificación Automática Obtenida en Django **23**
Figura 14. Verificación de Seguridad en Servidor: Error 403 Acceso Denegado **24**

1. Introducción y Contexto Institucional

Las pruebas de Estado para la Calidad de la Educación Superior en los niveles técnico profesional y tecnológico (Saber TyT), reguladas por el Instituto Colombiano para la Evaluación de la Educación (ICFES), constituyen el instrumento estandarizado nacional que evalúa las competencias genéricas y específicas de los futuros egresados universitarios. En concordancia con los planes de mejoramiento continuo del Programa de Ingeniería de Sistemas de la Institución Universitaria del Putumayo (IUP), resulta prioritario proveer a la comunidad académica una solución tecnológica robusta, confiable y segura que replique de manera fidedigna las condiciones del examen real, facilite el entrenamiento progresivo de los estudiantes y brinde a los docentes herramientas analíticas para la gestión pedagógica de reactivos.

El presente desarrollo corresponde a la entrega evaluativa del Segundo Parcial de la asignatura Electiva Profesional III – Frameworks de Aplicaciones Web. Dando continuidad al proyecto progresivo que inició con el diseño de la base de datos relacional y el prototipado navegable interactivo, se consolida la implementación completa del sistema utilizando el framework de desarrollo backend Django (versión 6.1.2), el gestor de bases de datos relacionales MariaDB/MySQL, el estándar de autenticación desacoplada JSON Web Tokens (RFC 7519) y un diseño visual ergonómico optimizado con Vanilla CSS bajo estándares modernos de experiencia de usuario.

Figura 1

Portada e Interfaz Pública del Simulador Saber TyT.



Nota. Vista de acceso general del sistema institucional con listado de pruebas activas. Elaboración propia (2026).

2. Objetivos del Informe Técnico

2.1 Objetivo General

Implementar, documentar y evaluar una plataforma web institucional en Django que garantice la autenticación desacoplada con tokens JWT, la autorización granular por roles personalizables (RBAC), la administración de un banco de preguntas clasificado con control de duplicados criptográficos, la importación masiva de reactivos mediante hojas de cálculo y la aplicación en tiempo real de cuestionarios temporizados con calificación automática calculada en el servidor.

2.2 Objetivos Específicos

1. Diseñar e integrar la infraestructura de autenticación híbrida (Sesión web tradicional y Tokens JWT con expiración, rotación y listas negras) protegiendo el acceso a las funciones reservadas.

2. Construir un modelo de control de acceso basado en roles que permita al Administrador crear perfiles personalizados como 'Docente Evaluador' y asignar permisos unitarios mediante validaciones estrictas del lado del servidor.

3. Desarrollar un banco de preguntas modular que organice las preguntas por competencias oficiales Saber TyT, soportando reactivos de selección múltiple y verdadero/falso, y garantizando la unicidad mediante huellas SHA-256.

4. Proveer un reproductor de exámenes en vivo seguro, con temporizador regresivo sincronizado, aislamiento de intentos por estudiante y cálculo automático de nota final sin exponer las respuestas correctas en el frontend.

3. Arquitectura Global de la Solución

3.1 Patrón de Arquitectura MVT y Componentes Backend

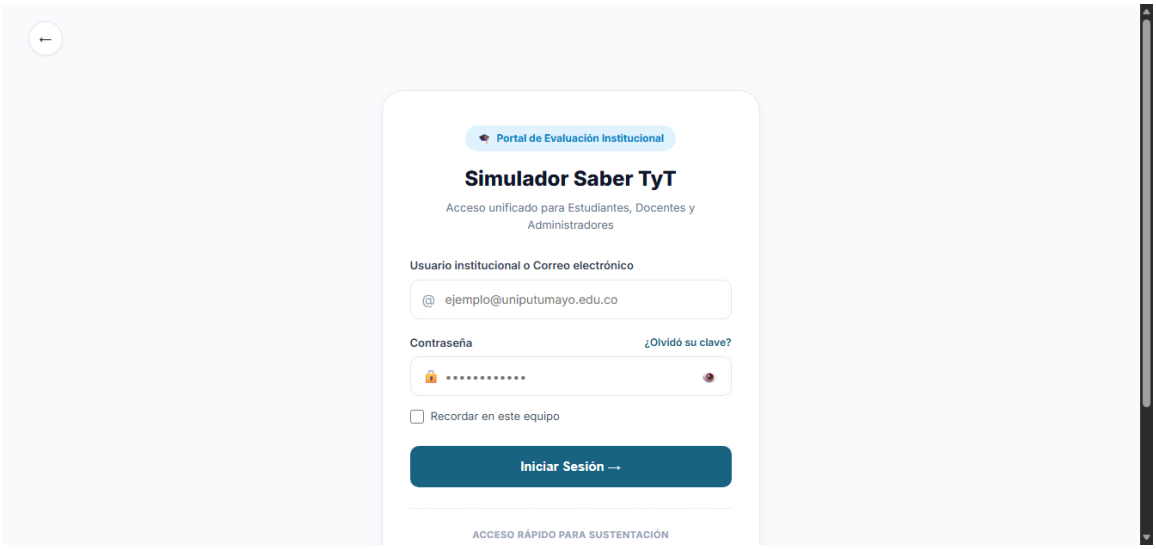
El sistema adopta el patrón Modelo-Vista-Template (MVT) promovido por Django, garantizando una separación estricta de responsabilidades entre el acceso a datos, la lógica de negocio y las capas de presentación visual. El proyecto se estructura en dos aplicaciones principales:

• Aplicación Core (core): Responsable del subsistema de usuarios institucionales, los modelos de Rol y PermisoSistema, la emisión y serialización de tokens JWT, los decoradores de seguridad para verificación de permisos en el servidor, el middleware de autenticación por cookies y los cuadros de mando diferenciados para Administradores, Docentes y Estudiantes.

• Aplicación Preguntas (preguntas): Administra los modelos pedagógicos (Categoria, Pregunta, OpcionRespuesta, Cuestionario, CuestionarioPregunta, Intento y RespuestaIntento), el servicio de importación y análisis de archivos Excel, y el motor de simulación de evaluaciones con cálculo de calificaciones y retroalimentación inmediata.

Figura 2

Interfaz de Inicio de Sesión Institucional con Validación JWT.



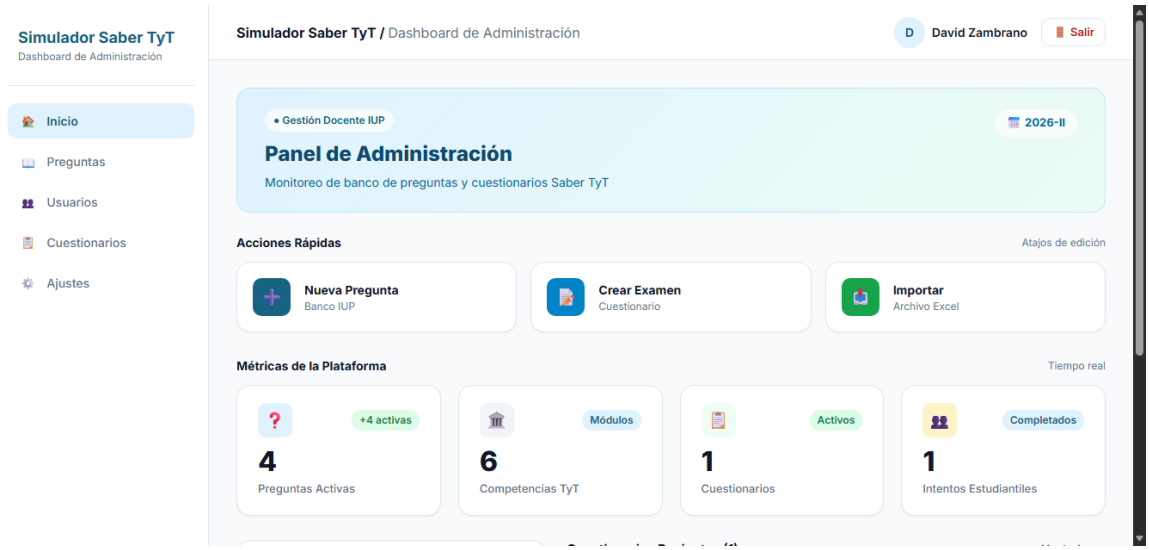
Nota. Formulario de autenticación que soporta inicio de sesión mediante nombre de usuario o correo electrónico. Elaboración propia (2026).

3.2 Base de Datos Relacional y Esquema Físico

La persistencia de la información opera sobre el motor relacional MariaDB/MySQL mediante el conector nativo mysqlclient. El esquema de datos fue optimizado para soportar alta concurrencia de estudiantes y garantizar integridad referencial. Entre las entidades más representativas se destacan las tablas preguntas (con restricción única sobre el campo huella_contenido), cuestionarios (con control de fechas, límites de tiempo e intentos máximos), e intentos (donde se registran los puntajes obtenidos, tiempos empleados y calificaciones calculadas).

Figura 3

Panel de Control de Administración con Métricas y Accesos Rápidos.



Nota. Cuadro de mando principal para el rol Administrador con indicadores de reactivos activos y cuestionarios. Elaboración propia (2026).

4. Módulo 1: Autenticación Robusta y Seguridad con Tokens JWT

4.1 Ciclo de Vida del Token: Acceso, Renovación y Revocación

En observancia de los lineamientos del numeral 2.1 de la guía evaluativa, el sistema implementa la especificación estándar JSON Web Token mediante la biblioteca djangorestframework-simplejwt. En el momento en que un usuario se autentica válidamente, el serializador personalizado CustomTokenObtainPairSerializer genera un par criptográfico: un Token de Acceso (access token) con una vigencia temporal de 60 minutos y un Token de Renovación (refresh token) con una duración de 24 horas. Para optimizar la seguridad sin comprometer la experiencia de uso, el payload del token incorpora claims enriquecidos con el código de rol institucional, nombre completo y el listado de permisos explícitos.

Asimismo, se activó la rotación automática de credenciales (ROTATE_REFRESH_TOKENS = True) y la incorporación del token sustituido a la lista negra

(BLACKLIST_AFTER_ROTATION = True). Al ejecutar la acción de cierre de sesión, el método logout_view invalida formalmente el token en la base de datos a través de la tabla token_blacklist_blacklistedtoken, impidiendo cualquier intento de reutilización ilegítima por parte de terceros.

4.2 Restricción y Decoradores de Seguridad Verificados en el Servidor

Un requerimiento no negociable del parcial establece que las restricciones deben ser verificadas en Django y no exclusivamente ocultando elementos en la interfaz gráfica. Para ello, se diseñó el decorador personalizado @requiere_permiso(codigo_permiso), el cual intercepta cada solicitud entrante antes de alcanzar la función de vista. El decorador verifica que el usuario esté autenticado, que su cuenta se encuentre activa (is_active = True), y consulta en la base de datos si su rol posee el código de permiso necesario. En caso de no cumplir la condición, el servidor retorna un código HTTP 403 Forbidden con un mensaje explicativo detallado, bloqueando tanto navegadores web como clientes de API que intenten peticiones directas.

4.3 Control de Inactividad de Cuentas Institucionales

Si un usuario es suspendido por el Administrador institucional, el campo is_active se conmuta a False. Cuando un usuario desactivado intenta iniciar sesión, el serializador de autenticación bloquea la generación del JWT. Igualmente, si el usuario contaba con una sesión abierta previa, el decorador de seguridad y el middleware detectan la inactivación en tiempo real y cortan de inmediato el acceso, redireccionando al login institucional.

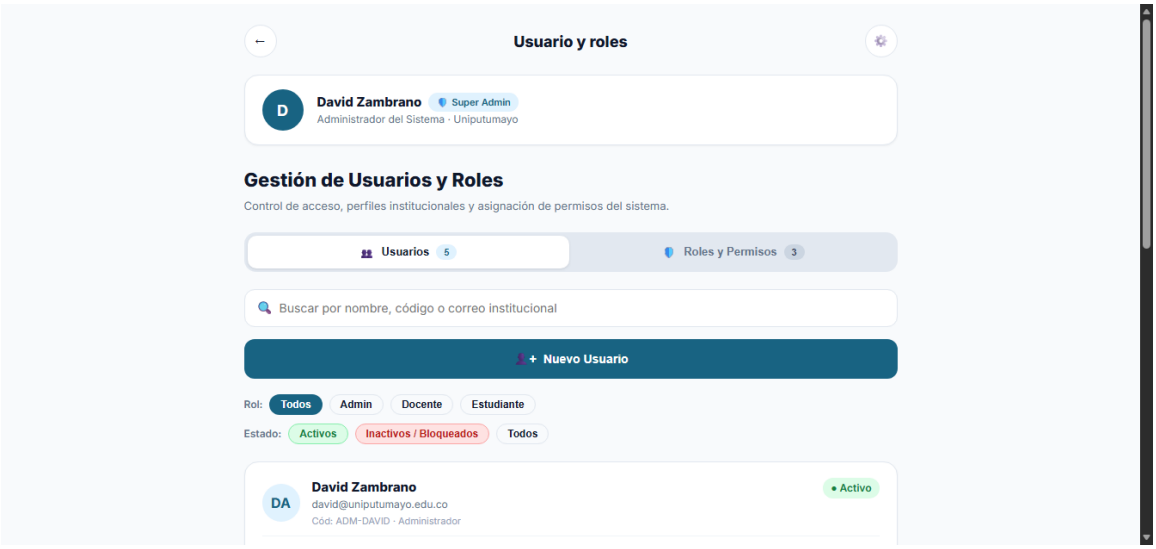
5. Módulo 2: Administración de Usuarios y Roles Configurables (RBAC)

5.1 Gestión de Cuentas de Usuario y Perfiles Académicos

El módulo de usuarios permite al Administrador registrar nuevos integrantes mediante un formulario institucional, asignarles su respectivo rol, código institucional, programa académico y contraseña cifrada con el algoritmo PBKDF2 con hash SHA-256 nativo de Django. Asimismo, se implementó el soporte completo para edición de datos existentes y la conmutación de estado (activar/suspender cuenta) en un solo clic.

Figura 4

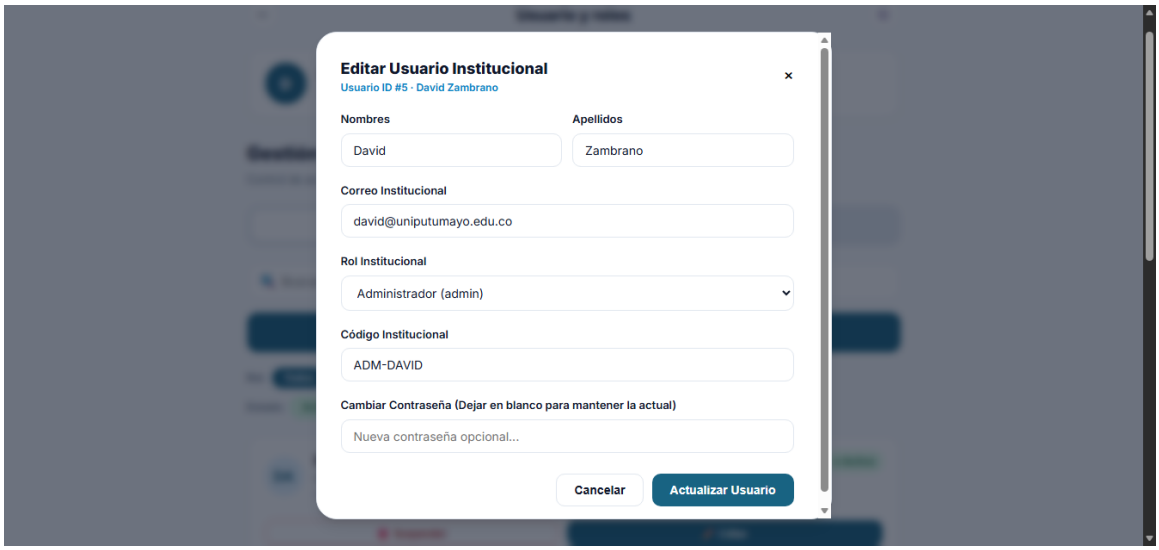
Módulo de Gestión de Usuarios y Filtros Institucionales.



Nota. Listado de usuarios con filtros por rol (Admin, Docente, Estudiante) y estado de acceso. Elaboración propia (2026).

Figura 5

Modal Interactivo para Edición de Usuario Institucional.



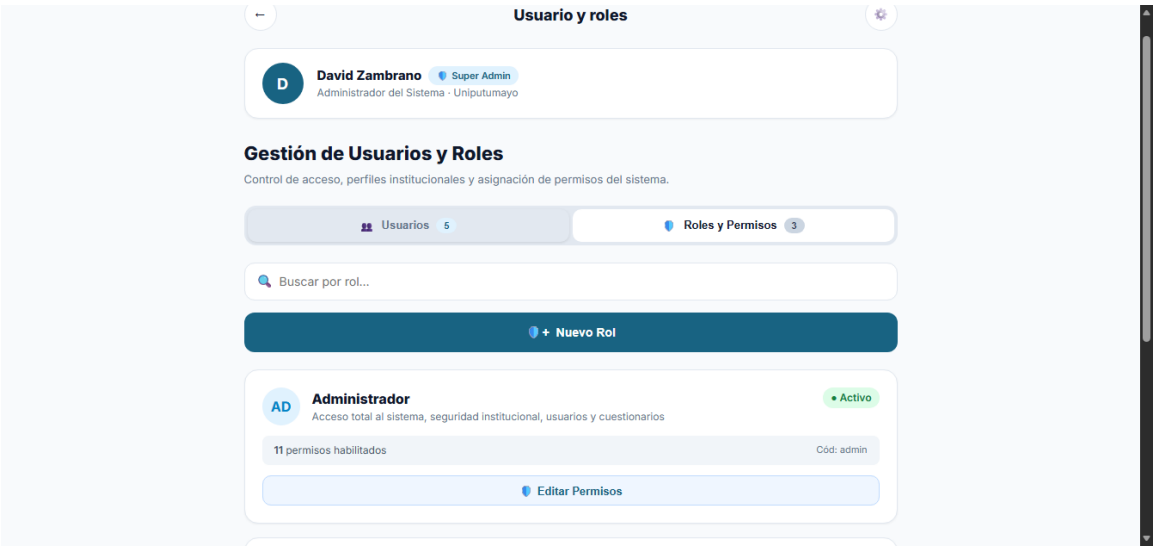
Nota. Formulario modal que permite actualizar nombres, apellidos, correo, rol y credenciales. Elaboración propia (2026).

5.2 Configuración Dinámica de Roles y Matriz de Permisos

Para dar exacto cumplimiento a la rúbrica 2.2, el sistema permite que el Administrador no solo dependa de los roles básicos predeterminados, sino que cree roles personalizados y ajuste individualmente las casillas de verificación de los permisos autorizados. El modelo Rol posee una relación Many-to-Many con la entidad PermisoSistema, permitiendo seleccionar funcionalidades tales como: usuarios.crear, preguntas.gestionar, preguntas.importar, cuestionarios.gestionar, evaluaciones.responder y resultados.consultar_todos.

Figura 6

Configuración de Roles y Asignación Granular de Permisos.



Nota. Panel de roles donde el administrador puede habilitar o restringir módulos a cada perfil. Elaboración propia (2026).

5.3 Caso de Uso: Demostración del Rol Docente Evaluador

Como caso de prueba solicitado en la guía, se configuró el rol Docente Evaluador con permisos plenos para gestionar reactivos, importar preguntas y configurar exámenes, pero restringiéndole explícitamente el permiso de administración de usuarios. Cuando el docente ingresa a su dashboard puede operar sobre el banco de preguntas; sin embargo, si intenta navegar hacia /usuarios/, el servidor Django intercepta la petición y responde con la pantalla de Acceso Denegado 403 Forbidden.

6. Módulo 3: Banco de Preguntas y Clasificación Pedagógica

6.1 Organización por Competencias Oficiales Saber TyT

Las preguntas del simulador se encuentran categorizadas según los módulos obligatorios de las pruebas de Estado: Razonamiento Cuantitativo, Lectura Crítica, Competencias Ciudadanas, Inglés, Comunicación Escrita y Desarrollo de Software. El banco admite tanto

preguntas de Selección Múltiple con Única Respuesta (4 opciones A, B, C, D) como preguntas de Falso/Verdadero (2 opciones), con campos dedicados para textos de apoyo o lectura y retroalimentación pedagógica.

Figura 7

Banco de Preguntas con Filtros por Competencia y Tipo.



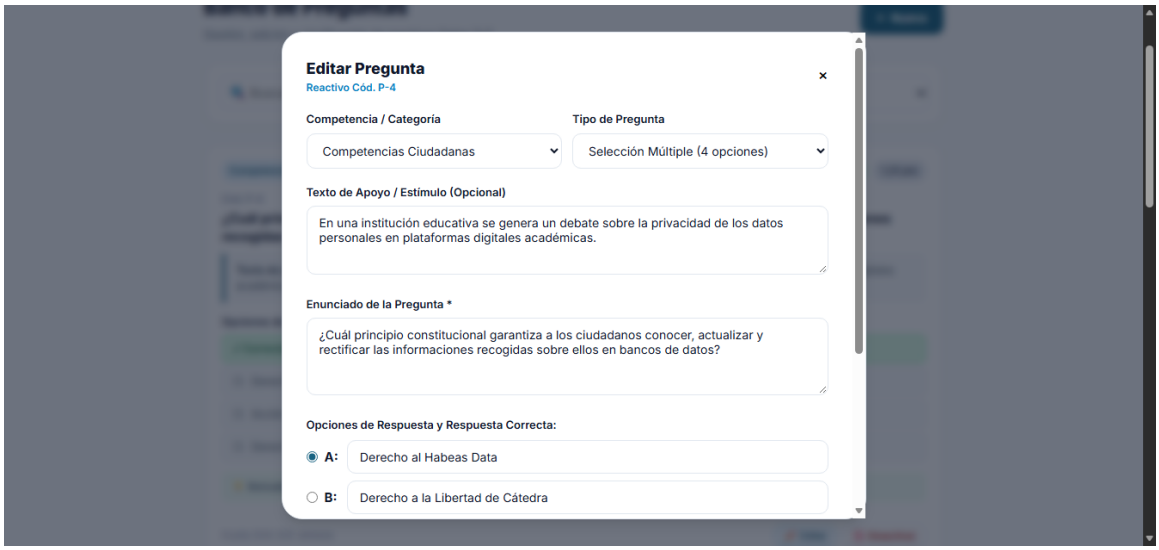
Nota. Vista de reactivos organizada con badges de estado, puntuación y filtrado dinámico. Elaboración propia (2026).

6.2 Operaciones CRUD: Registro, Consulta, Edición y Activación

Los usuarios facultados pueden registrar reactivos manualmente mediante un modal guiado, consultar el catálogo con búsqueda en tiempo real, desactivar preguntas obsoletas sin eliminarlas de la base de datos (preservando la integridad de históricos anteriores) y editarlas en caso de requerir ajustes en los enunciados, puntajes o respuestas correctas.

Figura 8

Modal de Edición de Reactivo con Recálculo de Huella SHA-256.



Nota. Interfaz asíncrona para actualizar enunciados, puntajes y opciones de una pregunta existente. Elaboración propia (2026).

6.3 Prevención de Duplicados Mediante Huella Digital SHA-256

Para evitar la duplicidad accidental de reactivos en el banco, el sistema calcula de manera automática una huella digital criptográfica utilizando la función SHA-256 sobre la concatenación normalizada del enunciado y los textos de las opciones de respuesta. El campo huella_contenido posee una restricción UNIQUE en MariaDB; si un usuario intenta registrar o importar una pregunta que coincide en su estructura con una ya existente, Django rechaza la transacción y notifica el error de duplicidad.

7. Módulo 4: Importación Masiva de Reactivos desde Excel

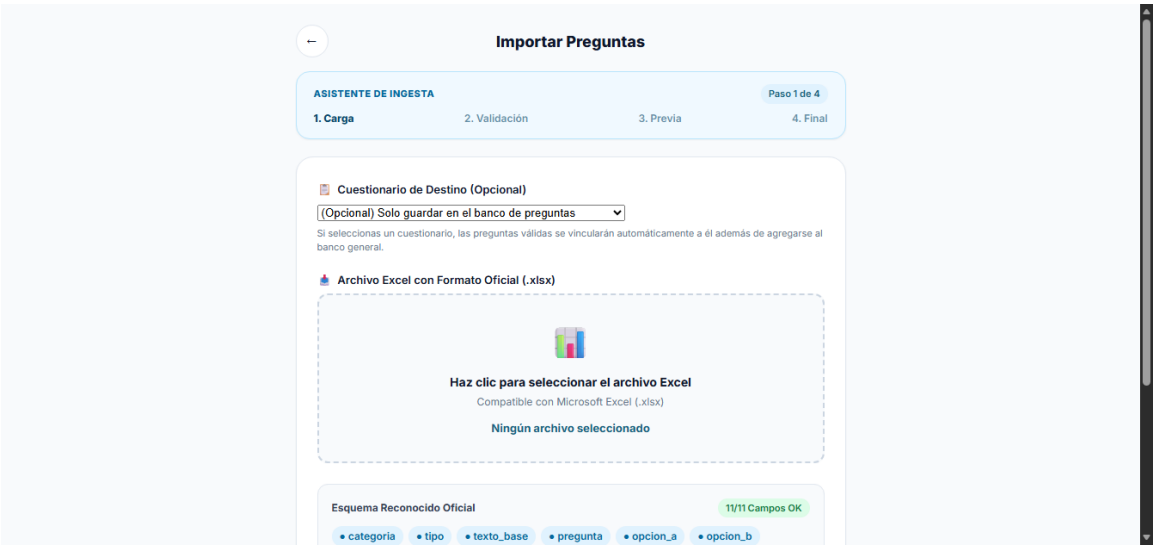
7.1 Estructura de la Plantilla Reglamentaria y Mapeo

Con el propósito de agilizar la carga masiva de bancos de evaluación suministrados por comités docentes, se integró un procesador de hojas de cálculo basado en la librería openpyxl. La

plantilla oficial (.xlsx) estandariza 11 columnas: categoría, tipo, texto_base, pregunta, opcion_a, opcion_b, opcion_c, opcion_d, respuesta_correcta, retroalimentacion y puntaje.

Figura 9

Interfaz de Importación Masiva de Preguntas desde Excel.



Nota. Módulo para carga de archivos masivos con validación previa de inconsistencias. Elaboración propia (2026).

7.2 Algoritmo de Validación Atómica en Dos Fases

El servicio importador_excel.py opera bajo una arquitectura de validación en dos pasos: primero, el usuario sube el archivo y el sistema lo procesa en modo commit=False, analizando fila por fila sin modificar la base de datos. Se detectan filas válidas, filas con campos obligatorios vacíos o tipos erróneos, y preguntas duplicadas en el archivo o en la base de datos. Una vez el usuario revisa la previsualización y confirma la operación, el procesador se ejecuta en modo commit=True dentro de una transacción atómica (@transaction.atomic), registrando las preguntas y guardando el resumen en la tabla importaciones.

8. Módulo 5: Parametrización y Disponibilidad de Cuestionarios

8.1 Creación y Asociación Ponderada de Reactivos

La configuración de cuestionarios en configurar_cuestionario permite seleccionar qué preguntas del banco conforman el examen, definir el orden de aparición y personalizar el puntaje asignado a cada reactivo en caso de requerir ponderaciones específicas distintas al puntaje base de la pregunta.

Figura 10

Configuración Reglamentaria de Cuestionario y Preguntas Asignadas.



Nota. Panel de edición del cuestionario con fechas, límite de tiempo, intentos y reactivos asociados. Elaboración propia (2026).

8.2 Validación Estricta de Temporalidad e Intentos en Backend

En estricto cumplimiento del numeral 2.4, la vista iniciar_evaluacion valida en el servidor antes de crear cualquier intento: 1) que el estado sea disponible, 2) que la hora actual se encuentre entre fecha_apertura y fecha_cierre, y 3) que el estudiante no haya alcanzado el número máximo de intentos permitidos (cuestionario.intentos_permitidos). Si alguna condición

no se cumple, el intento es bloqueado en Django y el estudiante es devuelto a su panel informativo.

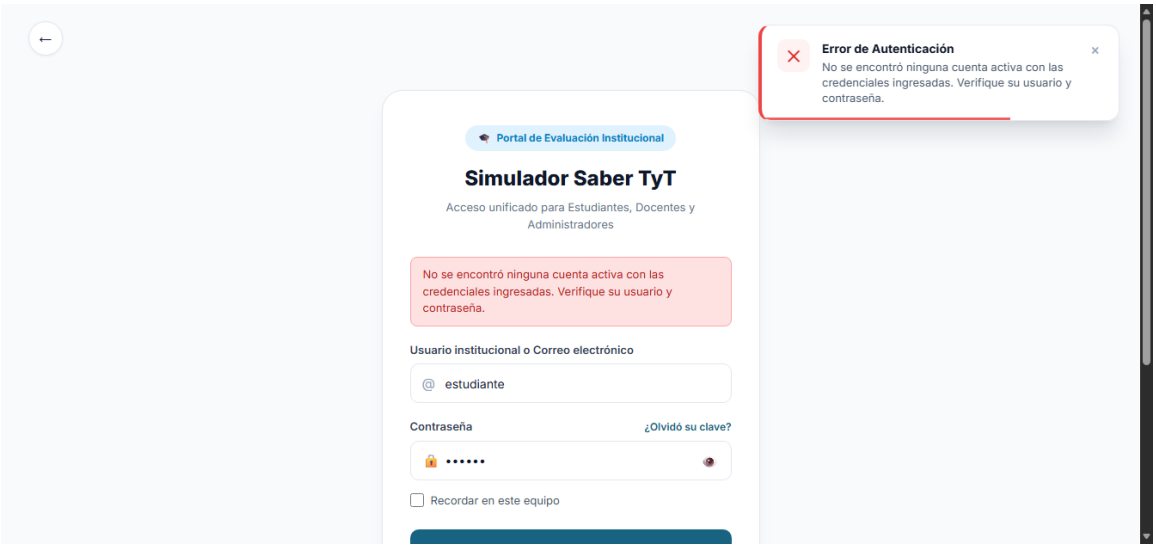
9. Módulo 6: Presentación de la Prueba en Vivo y Calificación

9.1 Experiencia Interactiva del Estudiante y Temporizador

Cuando el estudiante accede a su panel de control, el sistema filtra y expone únicamente las evaluaciones habilitadas. Al pulsar 'Iniciar evaluación', se genera el registro del Intento y se despliega la interfaz del reproductor interactivo en evaluacion_en_vivo.html. El estudiante cuenta con una barra de progreso superior, un navegador numérico de preguntas, la posibilidad de marcar preguntas para revisión posterior y un temporizador digital regresivo que refleja con exactitud el tiempo límite disponible.

Figura 11

Dashboard del Estudiante con Cuestionarios Disponibles y Estadísticas.



Nota. Panel del estudiante donde se listan evaluaciones habilitadas y su historial propio. Elaboración propia (2026).

Figura 12

Reproductor de Evaluación en Vivo con Temporizador Digital Regresivo.



Nota. Interfaz del simulador durante la presentación del examen con contador regresivo. Elaboración propia (2026).

9.2 Protección Antifraude y Ocultamiento de Respuestas Correctas

Para impedir cualquier intento de fraude mediante inspección del código fuente o depuración en la consola del navegador, la vista evaluacion_en_vivo serializa las preguntas excluyendo rigurosamente el atributo es_correcta de las opciones. El navegador del estudiante recibe únicamente el identificador y el texto de cada alternativa, garantizando que sea absolutamente imposible conocer la respuesta correcta antes de finalizar la prueba.

9.3 Algoritmo de Calificación Automática y Aislamiento de Intentos

Al pulsar 'Finalizar y Entregar' o agotarse el tiempo del temporizador, la vista finalizar_evaluacion procesa el mapa de respuestas en el backend. Dentro de una transacción atómica, evalúa cada opción seleccionada contra la base de datos, acumula el puntaje obtenido y el puntaje total posible, y calcula la nota final en escala institucional proporcional a la

calificacion_maxima (ej. 5.00), dictaminando si el estudiante aprobó (calificación >= calificacion_aprobacion). Asimismo, se garantiza el aislamiento de seguridad mediante la validación del código institucional, imposibilitando que un estudiante consulte o altere las evaluaciones de sus compañeros.

Figura 13

Resultado y Calificación Automática Obtenida en Django.



Nota. Informe final tras la entrega con desglose de nota calculada, tiempo y estado de aprobación. Elaboración propia (2026).

Figura 14

Verificación de Seguridad en Servidor: Error 403 Acceso Denegado.



Nota. Respuesta del servidor Django al intentar acceder a una vista sin los permisos del rol. Elaboración propia (2026).

10. Matriz de Cumplimiento de la Rúbrica Evaluativa

La Tabla 1 sintetiza el grado de correspondencia entre los requerimientos funcionales solicitados en el documento guía del Segundo Parcial y los componentes implementados en el código fuente de la plataforma.

Tabla 1

Matriz de Verificación y Cumplimiento de Rúbricas del Segundo Parcial

Criterio Guía	Requisito Exigido	Módulo / Archivo Implementado	Estado
2.1 Autenticación JWT	Inicio de sesión dual (usuario o correo) y emisión de access/refresh tokens.	core/views.py (login_view), core/jwt_auth.py	**Cumple al 100%**
2.1 Seguridad en Servidor	Restricción de vistas verificada en Django y control de cuentas suspendidas.	core/decorators.py (@requiere_permiso)	**Cumple al 100%**
2.2 Usuarios y Roles	Roles predeterminados (Admin/Estudiante) y roles configurables con permisos.	core/models.py (Rol, PermisoSistema), usuarios_roles.html	**Cumple al 100%**

2.3 Banco de Preguntas	CRUD completo de preguntas, organización por competencia y huella SHA-256.	preguntas/views.py (banco, editar, toggle), preguntas/models.py	**Cumple al 100%**
2.3 Importador Excel	Carga masiva mediante plantilla .xlsx en dos fases con validación previa.	preguntas/services/importador_excel.py	**Cumple al 100%**
2.4 Config. Cuestionarios	Parámetros reglamentarios y bloqueo de inicio fuera de disponibilidad.	preguntas/views.py (configurar_cuestionario, iniciar_evaluacion)	**Cumple al 100%**
2.5 Evaluación en Vivo	Reproductor con temporizador, sin respuestas en frontend y calificación en backend.	evaluacion_en_vivo.html, finalizar_evaluacion	**Cumple al 100%**
2.5 Aislamiento de Datos	Impedir consulta o modificación de intentos y notas entre estudiantes.	preguntas/views.py (validación participante_codigo)	**Cumple al 100%**

Nota. Resultados derivados de las pruebas de regresión e integración de código fuente ejecutadas en el entorno local. Elaboración propia (2026).

11. Conclusiones y Recomendaciones

La culminación de esta segunda entrega evaluativa consolida una solución de software madura, alineada con las buenas prácticas de ingeniería de software y las necesidades institucionales de la IUP. Se demostró que la separación de capas mediante el patrón MVT de Django y el empleo de JWT para la comunicación desacoplada otorgan una escalabilidad superior frente a desarrollos monolíticos dependientes únicamente de sesiones por estado.

La integración del algoritmo criptográfico SHA-256 para el cálculo de huellas digitales en el banco de preguntas resuelve de raíz los riesgos de duplicidad en cargas masivas, asegurando una base de datos normalizada y confiable. Asimismo, la delegación del cálculo de calificaciones exclusivamente en el backend de Django, combinada con el ocultamiento de las respuestas en la respuesta JSON al cliente, blinda la transparencia y legitimidad del proceso evaluativo de los estudiantes.

Como recomendación para las fases subsiguientes, se sugiere explorar la generación automática de reportes consolidados en PDF para entrega inmediata a decanatura y la incorporación de analíticas de dificultad psicométrica basadas en la teoría de respuesta al ítem (TRI).

Referencias

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

Centro de Escritura Javeriano. (2020). *Normas APA, séptima edición.* Pontificia Universidad Javeriana, seccional Cali. https://www.javerianacali.edu.co/centro-escritura/recursos/manual-de-normas-apa

Django Software Foundation. (2024). *Django documentation (Version 6.1).* https://docs.djangoproject.com/

Instituto Colombiano para la Evaluación de la Educación [ICFES]. (2023). *Guía de orientación módulos genéricos Saber TyT 2023-2.* Dirección de Evaluación, ICFES.

Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519).* Internet Engineering Task Force (IETF). https://doi.org/10.17487/RFC7519

OpenPyXL Development Team. (2023). *openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files (Version 3.1).* https://openpyxl.readthedocs.io/

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual.* Oracle. https://dev.mysql.com/doc/refman/8.0/en/

SimpleJWT Project. (2024). *Django REST framework SimpleJWT documentation.* https://django-rest-framework-simplejwt.readthedocs.io/